

LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL KE 7

Tree



Disusun Oleh:

Nama : Noufal Aji Prasetyo

NPM : 2340506059

Kelas : Rombel 3

Program Studi S1 Teknologi Informasi

Fakultas Teknik, Universitas Tidar

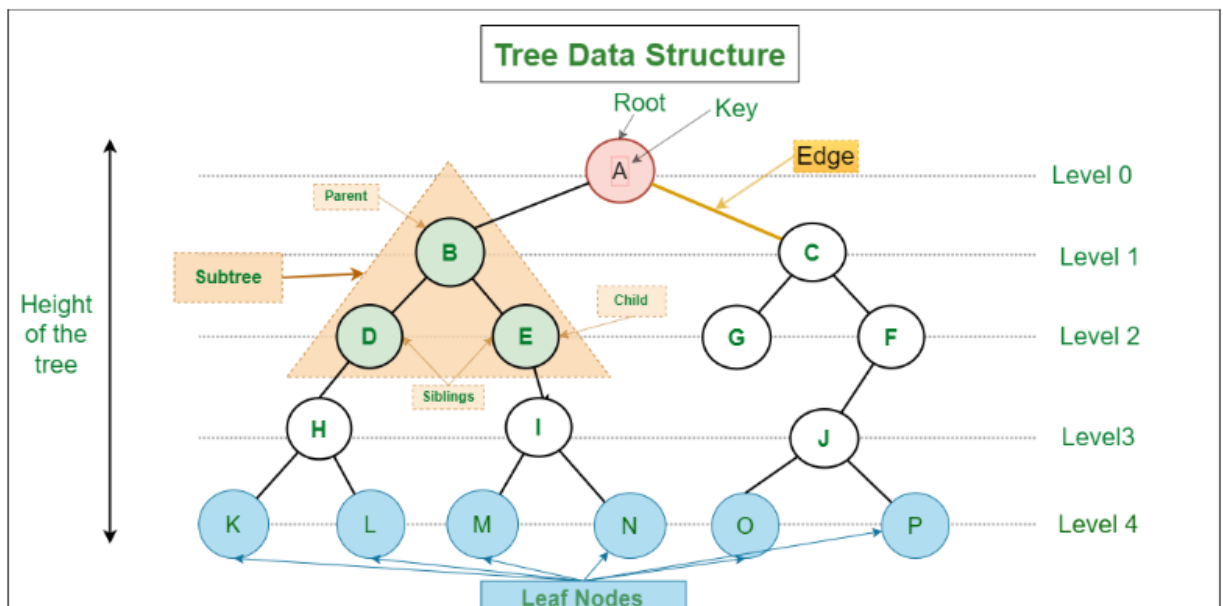
Genap 2024/2025

A. Tujuan Praktikum

Agar setiap mahasiswa tahu cara mengaplikasikan struktur data tree dalam python. Dan mahasiswa mampu memahami algoritma struktur data tree, macam-macam penerapan, dan fungsi pada setiap code

B. Dasar Teori

Struktur data tree adalah struktur data hierarki yang digunakan untuk mewakili dan mengatur data dengan cara mudah dinavigasi dan dicari. Ini adalah kumpulan node yang dihubungkan oleh tepi dan memiliki hubungan hierarki antar node. Node paling atas dari pohon disebut root, dan node dibawahnya disebut node anak. Setiap node dapat memiliki beberapa node anak, dan node anak ini juga dapat memiliki node anak sendiri sehingga membentuk struktur rekursif. Struktur data ini adalah metode khusus untuk mengatur dan menyimpan data di computer agar dapat digunakan secara lebih efektif. Ini terdiri dari node pusat, node struktur, dan sub-node, yang terhubung melalui tepi. Kita juga dapat mengatakan bahwa struktur data pohon memiliki akar, cabang, dan daun yang terhubung



Terminologi dasar dalam struktur data pohon

- Node induk: node yang merupakan pedahulu dari suatu node disebut node induk dari node tersebut. {B} adalah simpul induk dari {D,E}.
- Node anak: Node yang merupakan penerus langsung dari suatu node disebut node anak dari node tersebut. Contoh { D,E } adalah node anak dari {B}
- Node akar: simpul paling atas dari sebuah pohon atau simpul yang tidak memiliki simpul induk disebut simpul akar. {A} adalah simpul akar pohon. Pohon yang tidak kosong harus berisi tepat satu simpul akar dan tepat satu jalur dari akar ke semua simpul lain dalam pohon
- Node daun atau node eksternal: Node yang tidak memiliki node anak disebut node daun. {K,L,M,N,O,P,G} adalah simpul daun pada pohon
- Leluhur suatu node: Setiap node pendahulu pada jalur root ke node tersebut disebut leluhur dari node tersebut. {A,B} adalah node leluhur dari node {E}
- Keterunuan: Sebuah simpul adalah turunan dari simpul lain y jika dan hanya jika y merupakan nenek moyang dari y
- Saudara: anak-anak dari simpul induk yang sama disebut saudara kandung. {D, E} disebut saudara kandung
- Level suatu node: Jumlah tepi pada jalur dari node akar ke node tersebut. Node akar memiliki level 0
- Node internal: Node dengan setidaknya satu anak disebut node internal
- Tetangga node: Node induk atau anak dari node tersebut disebut tetangga dari node tersebut
- Subpohon: Setiap simpul pohon beserta turunannya.

Operasi dasar struktur data pohon

- **Buat:** Untuk membuat pohon di struktur data
- **Insert:** menyisipkan data ke dalam pohon
- **Search:** mencari dari tertentu di pohon untuk memeriksa apakah ada atau tidak
- **Melintas:**
 - **Preorder Traversal-** melakukan traveling pohon dengan cara pre-order dalam struktur data
 - **In order traversal-** melakukan traveling pada pohon secara berurutan
 - **Traversal pasca-pesanan-** melakukan perjalanan pada pohon dengan cara pasca-pesanan

Keuntungan

- Penawaran pohon pencarian efisien tergantung pada jenis pohon, dengan waktu pencarian rata-rata $O(\log n)$ untuk pohon seimbang seperti AVL
- Pohon memberikan representasi data secara hierarkis, sehingga memudahkan pengorganisasian dan navigasi informasi dalam jumlah besar
- Sifat pohon yang rekursif membuatnya mudah untuk dilintasi dan dimanipulasi menggunakan algoritma rekursif

Kekurangan

- **Unbalanced tree,** artinya tinggi pohon condong ke satu sisi sehingga dapat menyebabkan waktu pencarian menjadi tidak efisien
- Pohon menuntut lebih banyak kebutuhan ruang memori dibandingkan beberapa struktur data lain seperti array dan daftar terurut, terutama jika pohonnya sangat besar
- Implementasi dan manipulasi pohon bisa jadi rumit dan memerlukan pemahaman yang baik tentang algoritmanya

C. Hasil dan Pembahasan

```
class Node:
    def __init__(self, data):
        self.left = None
        self.right = None
        self.data = data

    def insert(self, data):
        #Compare the new value with the parent node
        if self.data:
            if data < self.data:
                if self.left is None:
                    self.left = Node(data)
                else:
                    self.left.insert(data)
            elif data > self.data:
                if self.right is None:
                    self.right = Node(data)
                else:
                    self.right.insert(data)
            else:
                self.data = data
        #Print the tree
        def PrintTree(self):
            if self.left:
                self.left.PrintTree()
            print(self.data),
            if self.right:
                self.right.PrintTree()
        #Use the insert method to add nodes
        root = Node(12)
        root.insert(6)
        root.insert(14)
        root.insert(3)
        root.PrintTree()
```

Gambar1.1

- Class node mendefinisikan sebuah kelas bernama node yang akan merepresentasikan simpul atau node dalam BTS
- Def __init__(self,data) metode kelas node yang akan dipanggil saat membuat objek node baru. Metode ini menginisialisasi node dengan data yang diberikan serta mengatur pointer untuk node kiri dan kanan ke None
- Self.left = None:pointer ke node anak kiri,awalnya diatur sebagai None
- Self.right = None pointer ke node anak kanan,awalnya diatur sebagai None
- Self.data = data, data yang dipegang oleh node ini
- Def insert(self,data) metode untuk menyisipkan data baru ke dalam BST
- If self.data ini memeriksa apakah node saat ini memiliki data. Jika iya maka dilakukan perbandingan untuk menentukan apakah data baru harus ditempatkan di anak kiri atau kanan dari node saat ini
- If data < self.data ini jika data baru lebih kecil dari node saat ini, maka akan ditempatkan di anak kiri

- If self.left is None ini jika tidak ada node anak kiri maka dibuatlah node baru dengan data tersebut dan diatur sebagai node anak kiri
- Self.left.insert(data) ini jika sudah ada node kiri, metode insert() dipanggil secara rekursif untuk menyisipkan data ke dalam subtree anak kiri
- Elif data>self.data jika data baru lebih besar dari data node saat ini maka akan ditempatkan di anak kanan
- If self.right is None jika tidak ada node anak kanan maka dibuatlah node baru dengan data tersebut dan diatur sebagai node anak kanan
- Self.right.insert(data) jika sudah ada node anak kanan, metode insert() dipanggil secara rekursif untuk menyisipkan data ke dalam subtree anak kanan
- Else: jika node saat ini kosong, maka node saat ini diinisialisasi dengan data baru
- Def printtree(self) ini metode untuk mencetak isi BST secara terurut
- If self.left jika ada node anak kiri, metode printtree() dipanggil secara rekursif untuk mencetak subtree anak kiri
- Self.right.printtree() untuk mencetak isi subtree anak kanan
- Root = node (12) membuat objek root yang merupakan akar dari BST dengan data awal 12
- Root.insert(6) untuk menyisipkan data ke 6 ke dalam BST dan seterusnya jika ada data yang akan dimasukkan
- Root.printtree() untuk mencetak isi BST secara terurut

```

class Node:
    def __init__(self, data):
        self.left = None
        self.right = None
        self.data = data

#Insert Node
def insert(self, data):
    if self.data:
        if data < self.data:
            if self.left is None:
                self.left = Node(data)
            else:
                self.left.insert(data)
        elif data > self.data:
            if self.right is None:
                self.right = Node(data)
            else:
                self.right.insert(data)
        else:
            self.data = data
#Print the Tree
def PrintTree(self):
    if self.left:
        self.left.PrintTree()
    print(self.data),
    if self.right:
        self.right.PrintTree()
#Inorder traversal
#Left -> Root -> Right
def inorderTraversal(self, root):
    res = []
    if root:
        res = self.inorderTraversal (root.left)
        res.append(root.data)
        res = res + self.inorderTraversal(root.right)
    return res

root = Node(27)
root.insert(14)
root.insert(35)
root.insert(10)
root.insert(19)
root.insert(31)
root.insert(42)
print(root.inorderTraversal(root))

```

Gambar1.2

- Node class ini adalah definisi dari kelas Node. Setiap node dalam BST memiliki dua pointer left dan right, yang menunjukkan ke node-node anak disebelah kiri dan kanan, serta sebuah attribute data yang menyimpan nilai dari node tersebut
- Metode insert ini digunakan untuk menyisipkan node baru ke dalam BST. Pertama, metode memeriksa apakah node saat ini memiliki data atau tidak. Jika tidak maka data yang disisipkan akan menjadi data node saat ini. Jika node saat ini memiliki data, maka metode membandingkan data baru dengan data node saat ini. Jika data baru lebih kecil dari node saat ini maka metode akan memeriksa apakah node kiri saat ini adalah Node. Jika iya maka node baru akan disisipkan sebagai anak kiri dari node saat ini. Logika yang sama berlaku untuk kasus ketika data baru lebih besar dari data node saat ini
- Metode inorderTraversal metode ini melakukan traversal inorder pada BST dan mengembalikan daftar (list) yang berisi elemen-elemen BST dalam urutan yang sesuai. Traversal inorder dilakukan dengan cara terlebih dahulu melakukan rekursif pada node kiri, kemudian menambahkan data dari node saat ini kedalam daftar, dan terakhir melakukan rekursif pada node kanan

- Inisialisasi BST sebuah objek root dibuat dengan nilai awal 27 kemudian, beberapa node lain disisipkan ke dalam BST menggunakan metode insert. Setelah itu, metode inordertraversal dipanggil pada node root untuk mencetak isi BST dalam urutan inorder

```
class Node:
    def __init__(self, data):
        self.left = None
        self.right = None
        self.data = data
#Insert Node
    def insert(self, data):
        if self.data:
            if data < self.data:
                if self.left is None:
                    self.left = Node(data)
                else:
                    self.left.insert(data)
            elif data > self.data:
                if self.right is None:
                    self.right = Node(data)
                else:
                    self.right.insert(data)
            else:
                self.data = data
#Print the Tree
    def PrintTree(self):
        if self.left:
            self.left.PrintTree()
        print(self.data),
        if self.right:
            self.right.PrintTree()
#Preorder traversal
#Root -> Left -> Right
    def PreorderTraversal (self):
        res = []
        if self:
            res.append(self.data)
            if self.left:
                res = res + self.left.PreorderTraversal()
            if self.right:
                res = res + self.right.PreorderTraversal()
        return res
root = Node(27)
root.insert(14)
root.insert(35)
root.insert(10)
root.insert(19)
root.insert(31)
root.insert(42)
print(root.PreorderTraversal())
```

Gambar 1.3

- Node class ini adalah definisi dari kelas Node. Setiap node dalam BST memiliki dua pointer left dan right, yang menunjukan ke node-node anak disebelah kiri dan kanan, serta sebuah attribute data yang menyimpan nilai dari node tersebut
- Metode insert ini sama dengan sebelumnya digunakan untuk menyisipkan node baru ke dalam BST
- Metode printTree ini juga sama dengan sebelumnya digunakan untuk mencetak isi BST dalam urutan inorder
- Metode preordertraversal metode ini melakukan traversal pre-order pada BST dan mengembalikan daftar list yang berisi elemen-elemen BST sesuai dengan urutan pre-order. Metode ini dimulai dengan menambahkan data dari node saat ini ke dalam daftar,

kemudian melakukan rekursif pada node kiri, dan terakhir melakukan rekursif pada node kanan

- Insialisasi BST seperti sebelumnya sebuah objek root dibuat dengan nilai awal 27. Kemudian,beberapa node lain disisipkan ke dalam BST menggunakan metode insert
- Cetak traversal pre-order metode preorderTraversal dipanggil pada node root untuk melakukan traversal pre-order dan mencetak daftar nilai BST dalam urutan tersebut

```
class Node:
    def __init__(self, data):
        self.left = None
        self.right = None
        self.data = data

#Insert Node
def insert(self, data):
    if self.data:
        if data < self.data:
            if self.left is None:
                self.left = Node(data)
            else:
                self.left.insert(data)
        elif data > self.data:
            if self.right is None:
                self.right = Node(data)
            else:
                self.right.insert(data)
        else:
            self.data = data

# Print the Tree
def PrintTree(self):
    if self.left:
        self.left.PrintTree()
    print(self.data),
    if self.right:
        self.right.PrintTree()
    #Postorder traversal
    #Left ->Right -> Root

def PostorderTraversal(self, root):
    res = []
    if root:
        res = self.PostorderTraversal(root.left)
        res = res + self.PostorderTraversal (root.right)
        res.append(root.data)
    return res

root = Node(27)
root.insert(14)
root.insert(35)
root.insert(10)
root.insert(19)
root.insert(31)
root.insert(42)
print (root.PostorderTraversal (root))
```

Gambar 1.4

- Node class ini adalah definisi dari kelas Node.Setiap node dalam BST memiliki dua pointer left dan right, yang menunjukan ke node-node anak disebelah kiri dan kanan, serta sebuah attribute data yang menyimpan nilai dari node tersebut
- Metode insert ini sama dengan sebelumnya digunakan untuk menyisipkan node baru ke dalam BST
- Metode printTree ini juga sama dengan sebelumnya digunakan untuk mencetak isi BST dalam urutan inorder
- Metode postorderTraversal metode ini melakukan traversal post-order pada BST dan mengembalikan daftar list yang berisi elemen-elemen BST sesuai dengan urutan

post-order. Metode ini dimulai dengan melakukan rekursif pada node kiri, kemudian melakukan rekursif pada node kanan, dan terakhir menambahkan data dari node saat ini ke dalam daftar

- Insialiasi BST seperti sebelumnya, sebuah objek root dibuat dengan nilai awal 27, kemudian beberapa node lain disisipkan ke dalam BST menggunakan metode insert
- Cetak traversal post-order metode ini dipanggil pada node root untuk melakukan traversal post-order dan mencetak daftar nilai BST dalam urutan tersebut

```
class Node:
s def __init__(self, data):
    self.data = data
    self.left = None
    self.righ = None
#inorder traversal of a binary tre
def inorde (temp):
    if(not temp):
        return
    inorde (temp.left)
    prin (temp.data, end=" ")
    inorde (temp.righ)
#function to delete the given deepest node (d_node) in binary tre
def deleteDeepest (root, d_node):
    q = []
    q.append(root)
    while (len(q)):
        e = q.pop(0)
        if temp is d_node:
            temp = None
            return
        if temp.righ:
            if temp.righ is d_node:
                temp.righ = None
                return
            else:
                q.append(temp.righ)
        if temp.left:
            if temp.left is d_node:
                temp.left = None
                return
            else:
                q.append(temp.left)
#function to delete element in binary tre
def deletion(root, key):
    if root == None:
        return None
    if root.left == None and root.righ == None:
        if root.key== key:
            return None
        else:
            return root
    key_node = None
    q = []
    q.append(root)
    temp = None
    while (len(q)):
        e = q.pop(0)
        if temp.data == key:
            key_node = temp
            if temp.left:
                q.append(temp.left)
            if temp.righ:
                q.append(temp.righ)
        if key_node == t)
            x = temp.dat
            key_node.data = x
            deleteDeepest (root, temp)
    return root
# Driver cod
if __name__ == "__main__" :
    root = Node(10)
    root.left = Node(11)
    root.left.left = Node(7)
    root.left.righ = Node(12)
    root.righ t= Node(9)
    root.righ .left = Node(15)
    root.righ .righ = Node(8)
    prin ("The tree before the deletion: ", end = "")
    inorde" (root)
    key = 11
    root = deletion (root, key)
    prin ();
    prin ("The tree after the deletion: ", end = "")
    inorde" (root)
    r
```

Gambar 1.5

- Node class ini adalah kelas yang mendefinisikan sebuah simpul dalam pohon biner. Setiap node memiliki tiga attribute. Data untuk menyimpan nilai, left untuk menunjukan ke simpul anak kiri, dan right untuk menunjukan ke simpul anak kanan

- Fungsi inorder ini adalah fungsi yang digunakan untuk melakukan traversal inorder pada pohon biner. Traversal inorder mengunjungi node-node dalam urutan kiri-akar-kanan, dan untuk setiap node nilainya dicetak
- Fungsi deletion fungsi utama untuk melakukan penghapusan elemen dalam pohon biner. Ini mencari elemen yang akan dihapus, kemudian menggantinya dengan nilai dari simpul terdalam. Setelah itu simpul terdalam tersebut dihapus
- Pengujian pada fungsi deletion ini sebuah pohon biner dibuat dengan simpul-simpul yang sudah ditentukan. Pohon ini kemudian dicetak sebelum dan setelah operasi penghapusan untuk memverifikasi keberhasilan operasi penghapusan

```
#Python3 function to search a given key in a given BST
class Node:
    s
    # Constructor to create a new node
    e
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        t

#A utility function to insert
#A new node with the given key in BST
def insert(node, key):

    # If the tree is empty, return a new node
    if node is None:
        return Node(key)

    # Otherwise, recur down the tree
    e
    if key < node.key:
        node.left = insert(node.left, key)
    elif key > node.key:
        node.right = insert(node.right, key)

    #Return the (unchanged) node pointer
    return node

#Utility function to search a key in a BST
def search(root, key):

    # Base Cases: root is null or key is present at root
    if root is None or root.key == key:
        return root

    #Key is greater than root's key
    if root.key < key:
        return search(root.right, key)

    #Key is smaller than root's key
    return search(root.left, key)

# Driver Code
if __name__ == '__main__':
    root = None
    root = insert(root, 50)
    insert(root, 30)
    insert(root, 20)
    insert(root, 40)
    insert(root, 70)
    insert(root, 60)
    insert(root, 80)

    #Key to be found
    d key = 6
    #Searching in a BST
    T if search(root, key) is None:
        print(key, "not found")
    else:
        print(key, "found")
    key = 60
    #Searching in a BST
    T if search(root, key) is None:
        print(key, "not found")
    else:
        print(key, "found")
    t
```

Gambar 1.6

- Kelas Node ini adalah kelas yang mendefinisikan sebuah simpul dalam pohon pencarian biner. Setiap simpul memiliki tiga attribute, key untuk menyimpan nilai

kunci, left untuk menunjukan ke simpul anak kiri, dan right untuk menunjukan ke simpul anak kanan

- Fungsi insert digunakan untuk menyisipkan simpul baru dengan nilai kunci tertentu ke dalam BST. Fungsi ini mencari tempat yang sesuai untuk menyisipkan simpul baru dalam BST, berdasarkan nilai kunci simpul yang ada saat ini
- Fungsi search ini adalah fungsi rekursif yang digunakan untuk mencari nilai kunci tertentu dalam BST. Fungsi ini mencari nilai kunci secara rekursif mulai root node. Jika nilai kunci yang dicari lebih besar dari nilai kunci root, pencarian dilanjutkan ke anak kanan, dan jika lebih kecil, pencarian dilanjutkan ke anak kiri. Jika nilai kunci ditentukan, fungsi mengembalikan node tempat nilai kunci ditentukan, jika tidak mengembalikan None
- Pengujian pada fungsi search sebuah pohon biner dibuat dengan beberapa simpul yang telah ditentukan. Kemudian dilakukan pencarian nilai kunci tertentu dalam BST yang telah dibuat. Jika nilai kunci ditentukan pesan found dicetak, jika tidak pesan not found dicetak

D. Latihan

```
class Node:
    def __init__(self, val=None):
        self.left = None
        self.right = None
        self.val = val

    def insert(self, val):
        if not self.val:
            self.val = val
            print(f"Inserted {val}")
            return

        if val < self.val:
            if self.left:
                self.left.insert(val)
            else:
                self.left = Node(val)
            print(f"Inserted {val}")
            return

        if self.right:
            self.right.insert(val)
        else:
            self.right = Node(val)
        print(f"Inserted {val}")

    def delete(self, val):
        if self is None:
            return self
        if val < self.val:
            if self.left:
                self.left = self.left.delete(val)
            else:
                return self.left
        elif val > self.val:
            if self.right:
                self.right = self.right.delete(val)
            else:
                return self.right
        else:
            if self.right is None:
                return self.left
            if self.left is None:
                return self.right
            min_larger_node = self.right
            while min_larger_node.left:
                min_larger_node = min_larger_node.left
            self.val = min_larger_node.val
            self.right = self.right.delete(min_larger_node.val)
            return self

    def inorder(self, vals):
        if self.left:
            self.left.inorder(vals)
        if self.val is not None:
            vals.append(self.val)
        if self.right:
            self.right.inorder(vals)
        return vals

    def preorder(self, vals):
        if self.val is not None:
            vals.append(self.val)
        if self.left:
            self.left.preorder(vals)
        if self.right:
            self.right.preorder(vals)
        return vals

    def postorder(self, vals):
        if self.left:
            self.left.postorder(vals)
        if self.right:
            self.right.postorder(vals)
        if self.val is not None:
            vals.append(self.val)
        return vals

def main():
    nums = [12, 6, 18, 19, 21, 11, 3, 5, 4, 24, 18]
    bst = Node()
    for num in nums:
        bst.insert(num)
    print("preorder:")
    print(bst.preorder([]))
    print("#")

    print("postorde ")
    print(bst.postorder([]))
    print("#")

    print("inorde ")
    print(bst.inorder([]))
    print("#")

    nums = [2, 6, 20]
    print("deleting " + str(nums))
    for num in nums:
        bst.delete(num)
    print("#")

main()
```

Gambar 1.7

- Class node mendefinisikan sebuah kelas python bernama node. Kelas ini akan mewakili

simpul dalam pohon biner atau BST

- Def init (self, val=None) adalah metode khusus yang disebut kontruktor. Ketika objek dari keelas node dibuat, metode ini akan dipanggil secara otomatis untuk menginisialisasi objek tersebut. Parameter val=None memberikan nilai default none untuk val
- Self.left = none, self.right=noen, dan self.val=val untuk menginisialisasi attribute dari objek yang dibuat. Self.left dan self.right merepresentasikan anak kiri dan anak kanan dari simpul. Selft.val menyimpan nilai dari simpul tersebut
- Def insert(self, val) adalah metode yang digunakan untuk memasukan nilai baru ke dalam tree. Metode ini menerima parameter val yang merupakan nilai yang akan dimasukkan
- Pada baris if not self.val memeriksa apakah nilai saat ini kosong. Jika iya maka nilai tersebut diisi dengan val dan pesan inserted dicetak. Kemduain metode ini segera Kembali
- Pada baris if val<self.val memiliki arti jika val kurang dari self.val nilai barus harus dimasukkan ke anak kiri. Jika anak kiri sudah ada metode insert dipanggil lagi secara rekursif untuk anak kiri. Jika tidak simpul anak kiri baru dibuat dengan nilai val dan pesan inserted dicetak
- Selajutnya baris if self.right memiliki arti jika val lebih besar dari self.val nilai baru harus dimasukkan ke anak kanan. Jika anak kanan suda ada metodeinsert dipanggil lagi secara rekursif untuk anak kanan.jika tidak simpul anak kanan baru dibuat dengan nilai val dan pesan inserted dicetak
- Pada baris def delete(self, val) adalahmetode yang digunakan untuk menghapus nilai tertentu dari tree. Metode ini menerima parameter val yang merupakan nilai yang akan dihapus
- Untuk baris if self is none untuk memriksa apakah simpul saat ini kosong. Jika ya metode segera mengembalikan simpul itu sendiri
- Untuk baris if val < self.val dan elif val>self.val memiliki arti jika val kurang dari self.val maka metode delete dipanggil rekursif untuk anak kiri dan jika val lebih besar dari self.val metode delete dipanggil rekursif untuk anak kanan
- Untuk baris else:if self.right is none dan if self.left isi none memiliki arti jika nilai yang ingin dihapus sama dengan nilai di simpul saat ini, maka ini adalah kasus dimana simpul yang akan dihapus adalah daun atau simpul dengan satu anak. Jika simpul yang akan

dihapus tidak memiliki anak, metode mengembalikan anak kiri. jika tidak memiliki anak kiri, metode mengembalikan anak kanan.

- Untuk baris `min_larger_node=self.right` dan baris `while` ini jika simpul yang akan dihapus memiliki dua anak, metode mencari nilai terkecil dibawah anak kanan simpul tersebut. Nilai ini kemudian menggantikan nilai simpul yang akan dihapus dan metode `delete` dipanggil untuk menghapus nilai terkecil itu dari anak kanan simpul tersebut
- Pada akhirnya metode mengembalikan simpul itu sendiri setelah operasi penghapusan selesai
- Pada baris `def inorder(self.val)` adalah metode yang digunakan untuk melakukan penelusuran inorder pada tree. Metode ini menerima parameter `vals` yang merupakan daftar untuk mengakumulasi nilai-nilai. Jika ada anak kiri, metode dipanggil rekursif untuk anak kiri, nilai dari simpul saat ini ditambahkan ke dalam daftar `vals`. jika ada anak kanan metode dipanggil rekursif untuk anak kanan. Dan pada akhirnya metode mengembalikan daftar nilai yang telah diurutkan secara inorder
- Untuk metode `preorder` dan `postorder` memiliki struktur yang serupa dengan metode `inorder`
- Pada baris `def main ()` adalah fungsi utama yang akan dieksekusi ketika skrip dijalankan
- Pada baris `nums` adalah daftar nilai yang akan dimasukkan ke dalam tree
- Pada baris `bst` adalah objek dari kelas `node` yang akan digunakan untuk mewakili tree
- Pada baris `for num in nums` melalui loop ini setiap nilai dari `nums` dimasukkan ke dalam tree menggunakan metode `insert`
- Baris `print ("preorder")` untuk mencetak hasil penelusuran preorder tree
- Baris `print ("postorder")` untuk mencetak hasil penelusuran postorder tree
- Baris `print ("inorder")` untuk mencetak hasil dari penelusuran inorder tree
- Baris `nums = [2,6,20]` adalah daftar nilai yang akan dihapus dari tree
- Untuk baris `print ("deleting" + str(nums))` ini mencetak nilai dari `nums` yang akan dihapus
- Pada baris `for num in nums` melalui loop ini setiap nilai dari `nums` dihapus dari tree menggunakan metode `delete`
- Baris `main ()` untuk memanggil fungsi `main()` untuk memulai eksekusi program

E. Kesimpulan

- Dari praktikum diatas kita bisa lebih memahami tentang Binary Search Tree, implemnetasi metode traversal,penambahan dan penghapusan nilai,rekursif,pemahaman tentang metode yang ada didalam python. Dengan demikian kita dapat memperdalam pemahaman struktur data BST dan implementasi algoritma dasar yang terkait dengannya.

F. Referensi

- <https://www.geeksforgeeks.org/introduction-to-trie-data-structure-and-algorithm-tutorials/?ref=shm>